

SYSTEM SERVICE MAKER

A C-Based Linux System Programming Utility

Course: CS-411 — System Programming

Date:

April 2026

Team Ahmad

Muhammad Ahmad • Marwan Rahim • Aleena Khan

Automate Linux Service Management with One Command

Table of Contents

01 What is a System Service?

02 The Problem — Manual Process

03 Our Solution

04 System Architecture

05 File Structure & Module Roles

06 marwan.c — Core Logic

07 aleena.c — System Integration

08 main.c — Controller & Logger

09 POSIX System Calls Used

10 fork() + execvp() — How It Works

11 What Happens in the Background

12 CLI Commands & Usage

13 Live Demo — Create a Service

14 Error Handling & Validation

15 Test Cases & Results

16 Why This Project is Better

17 Team Contributions

18 Conclusion & Viva Points

What is a System Service?

A program that runs silently in the background — always on, never visible.



WiFi Manager

Connects to network automatically on boot



Print Spooler

Waits for print jobs 24/7 in the background



SSH Server

Accepts remote connections without manual start

Managed by systemd

systemd is the master process manager of modern Linux (Ubuntu, Fedora, Debian). It reads `.service` files to know how to start, stop, restart, and monitor every background program. It is the first process started by the Linux kernel (PID 1) and controls everything that runs in the background.

The Problem — Manual Process

Registering a service manually requires 5 error-prone steps:

1

```
nano /etc/systemd/system/myapp.service
```

Write unit file by hand — one typo breaks everything

2

```
sudo cp myapp.service /etc/systemd/system/
```

Copy to the exact correct system directory

3

```
sudo systemctl daemon-reexec
```

Reload daemon so systemd detects the new file

4

```
sudo systemctl enable myapp
```

Enable so it starts on every boot

5

```
sudo systemctl start myapp
```

Manually start for current session

Our Solution

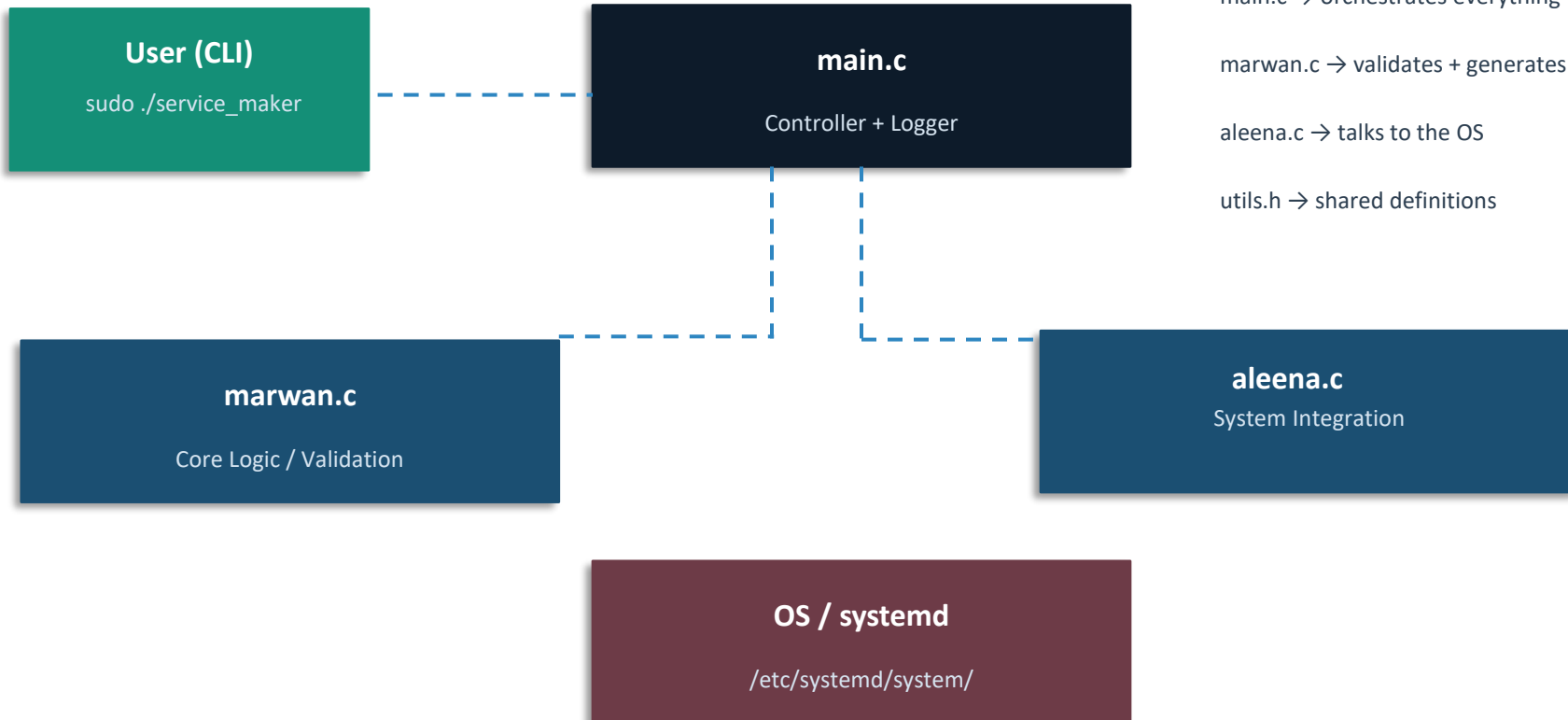
Replace ALL 5 manual steps with a single command:

```
sudo ./service_maker create myapp /usr/local/bin/myservice.sh
```

This one command automatically:

- ✓ Validates the service name and executable path
- ✓ Writes the .service unit file with correct systemd syntax
- ✓ Copies it to /etc/systemd/system/
- ✓ Reloads the systemd daemon
- ✓ Enables the service to auto-start on every boot
- ✓ Logs every operation with a timestamp to log.txt

System Architecture Overview



File Structure & Module Roles

project/

├─ main.c

Controller + CLI parser + Logger

├─ marwan.c

Validation + .service file writer

├─ aleena.c

fork()+exec() system integration

├─ utils.h

Shared macros, prototypes, consts

├─ Makefile

└─ log.txt (auto-created)

main.c

Owner: Ahmad

CLI parser, check_root(), write_log(), pipeline orchestrator

marwan.c

Owner: Marwan

validate_service_name(), validate_path(), create_service_file()

aleena.c

Owner: Aleena

install_service(), reload_daemon(), enable/start/stop/status

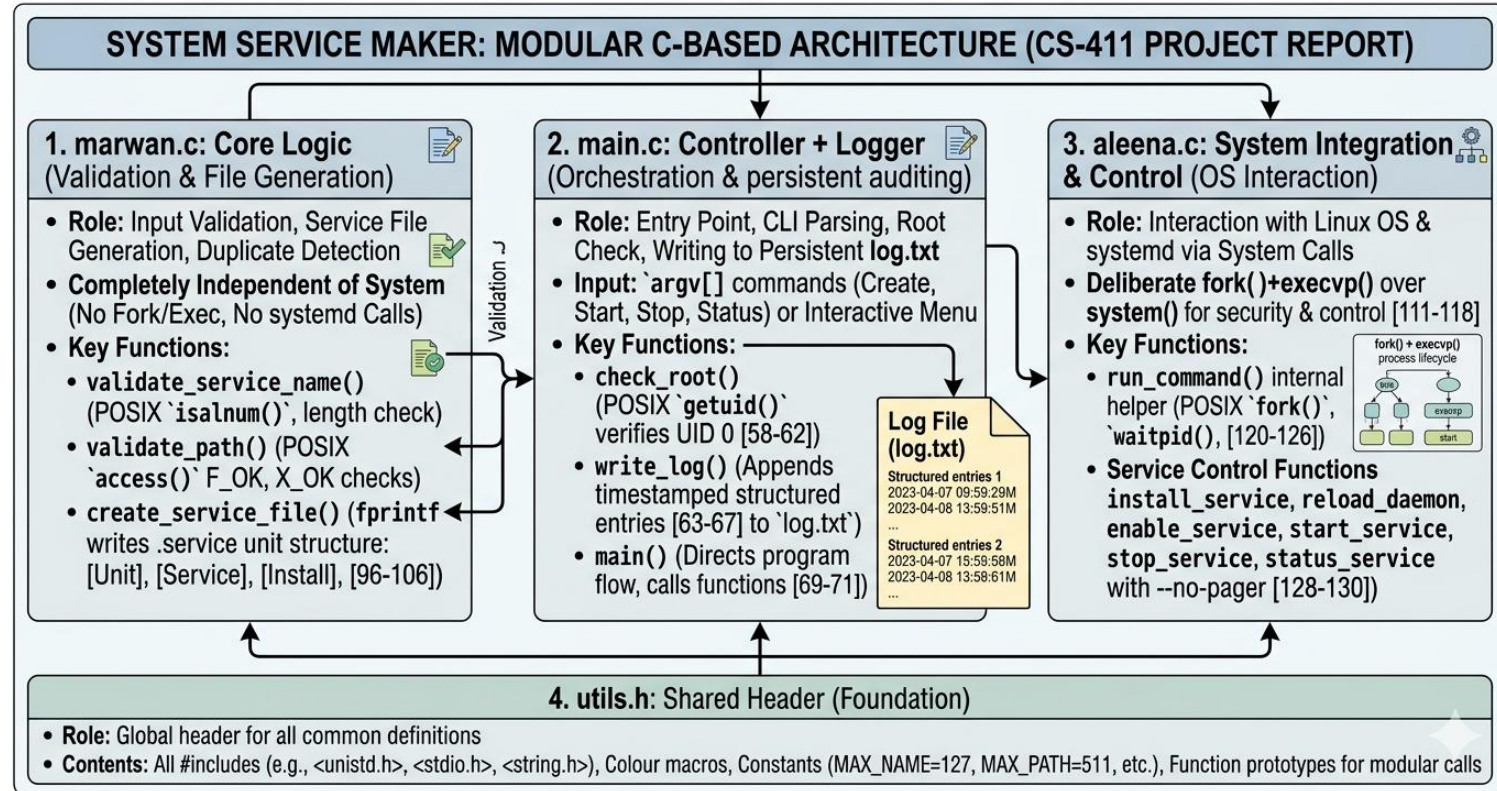
utils.h

Owner: Shared

All #includes, color macros, MAX constants, function prototypes

System Architecture Overview

FIGURE 1: MODULAR SYSTEM SERVICE MAKER ARCHITECTURE & FUNCTIONAL RESPONSIBILITIES



aleena.c — System Integration

All functions use `fork()` + `execvp()` — never `system()`. A shared `run_command()` helper handles the fork/wait pattern.

install_service()

```
cp name.service /etc/systemd/system/
```

reload_daemon()

```
systemctl daemon-reexec
```

enable_service()

```
systemctl enable <name>
```

start_service()

```
systemctl start <name>
```

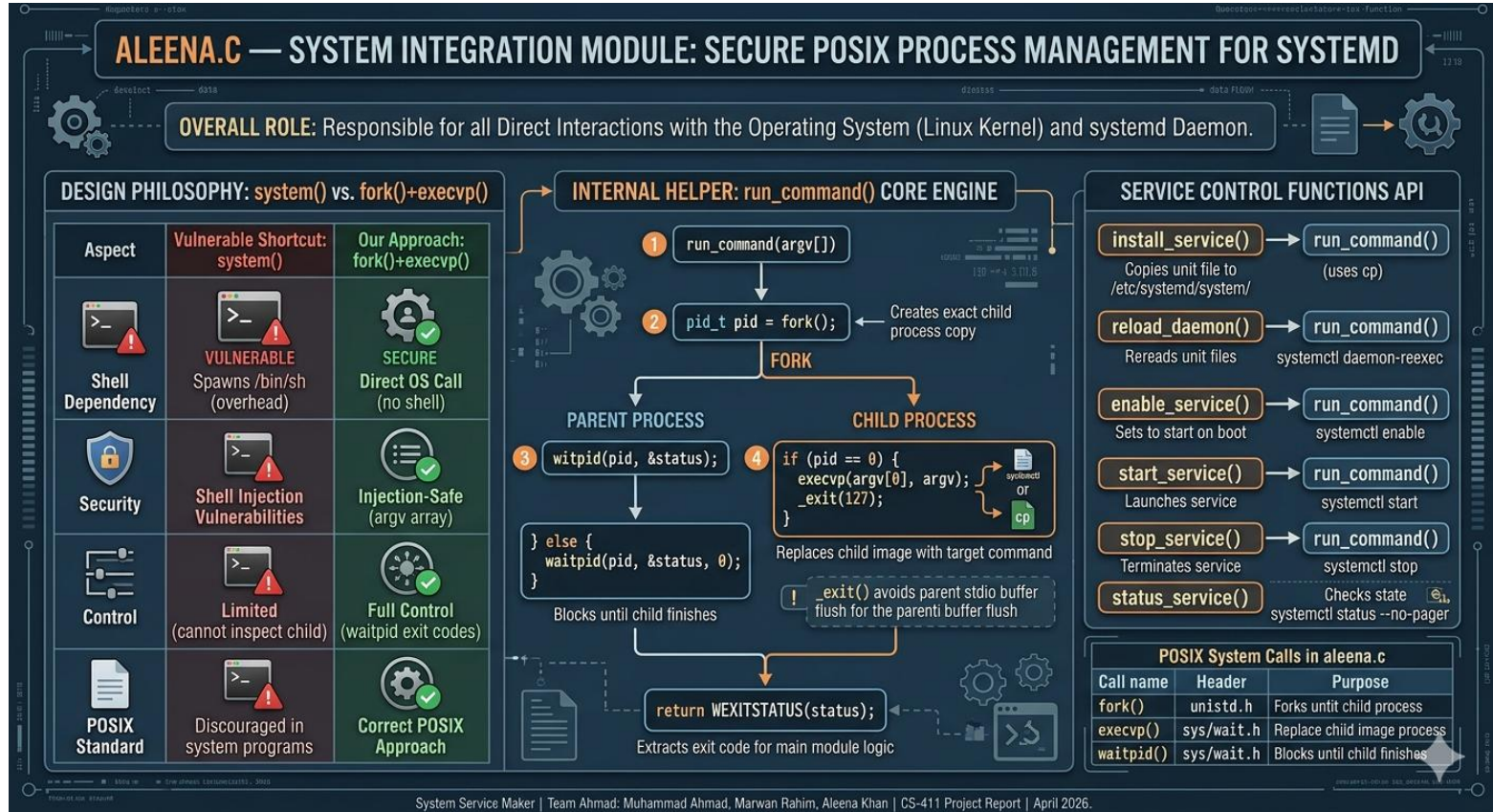
stop_service()

```
systemctl stop <name>
```

status_service()

```
systemctl --no-pager status <name>
```

aleena.c — System Integration



marwan.c — Core Logic

validate_service_name()

Loops through every character using `isalnum()`. Rejects: empty, >127 chars, spaces, special chars, leading dot.

Returns 1 (valid) or 0 (invalid)

```
if (!isalnum(c) && c!='-' && c!='_' &&
    c!='.')
    return 0; // invalid character found
```

validate_path()

Two POSIX `access()` calls: `F_OK` checks existence, `X_OK` checks execute permission. Direct kernel-level file check.

Returns 1 (valid) or 0 (invalid)

```
access(path, F_OK); // file exists?
access(path, X_OK); // is executable?
```

create_service_file()

Checks for duplicate in `/etc/systemd/system/` first. Then `fopen()` + `fprintf()` writes the complete `.service` unit file.

Returns 1 (success) or 0 (failure)

```
access("/etc/systemd/system/name.service",
    F_OK)
fp = fopen("name.service", "w");
```

marwan.c — Core Logic

CORE LOGIC: **marwan.c** MODULE (SYSTEM SERVICE MAKER)

 **MODULE OVERVIEW:** `marwan.c` (CORE LOGIC RESPONSIBILITIES)

CONTEXT  

 Independent of systemd / system calls

 No fork/exec

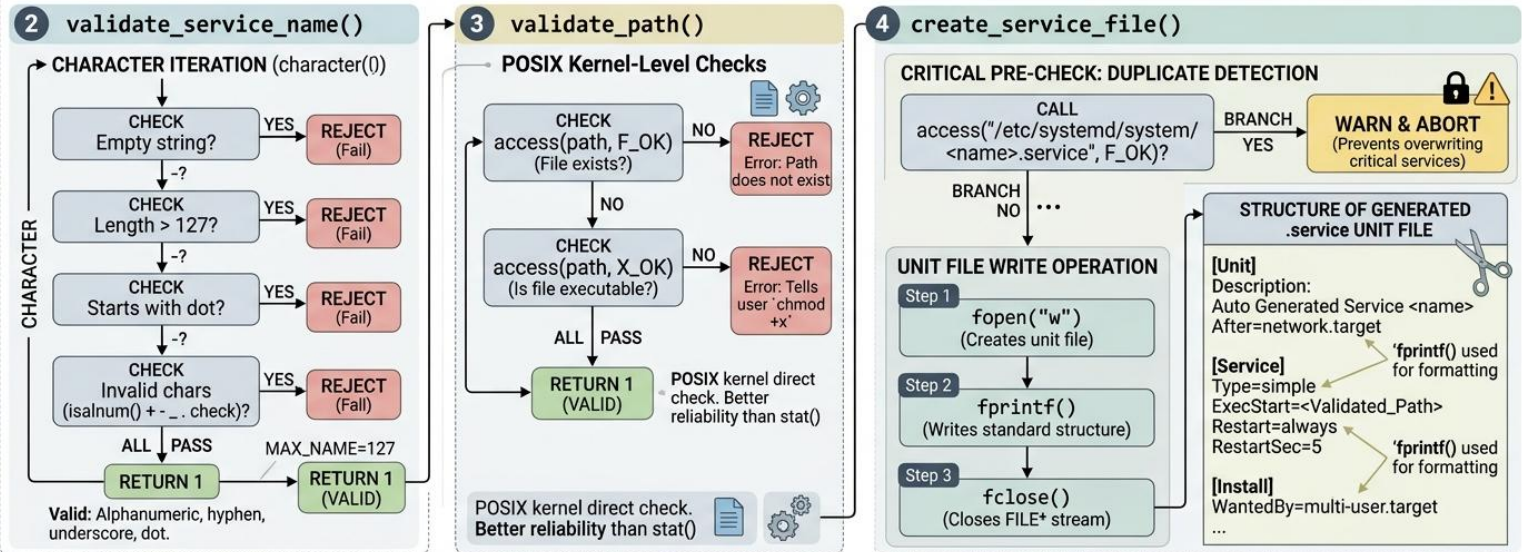
 Testable in isolation (no root required)

• INPUT VALIDATION (NAME & PATH)

• SERVICE FILE GENERATION

• DUPLICATE DETECTION

INPUT VALIDATION & SERVICE FILE GENERATION PIPELINE



Core Logic (marwan.c): A multi-stage pipeline ensuring robust validation, standard compliance, and system safety before any unit file is created.

main.c — Controller & Logger

`print_banner()`

Displays the project banner with team names on startup

`check_root()`

`getuid()==0` check — exits immediately if not running as root

`write_log()`

`fopen(log.txt, "a")` → timestamp → `fprintf` → `fclose`. Appends every operation.

`do_create()`

Calls: `validate_name` → `validate_path` → `create_file` → `install` → `reload` → `enable`

`do_start/stop/status()`

Validates name then delegates to matching `aleena.c` function

`interactive_menu()`

Shown when no CLI args given: numbered menu 1-5 for all operations

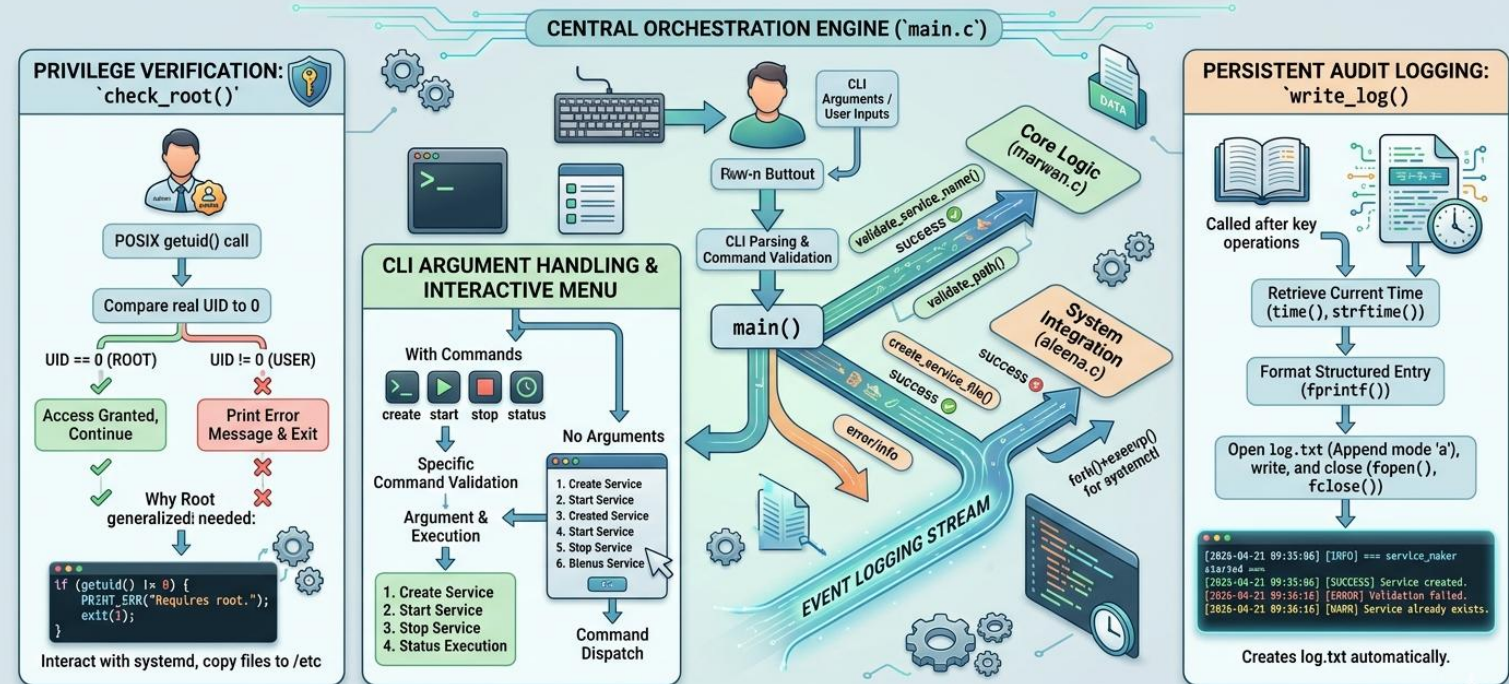
log.txt sample

```
[INFO  ] === started ===  
  
[INFO  ] Validating name  
  
[SUCCESS] Name is valid  
  
[SUCCESS] Path is valid  
  
[SUCCESS] File created  
  
[SUCCESS] Installed  
  
[SUCCESS] Daemon reloaded  
  
[SUCCESS] Enabled  
  
[SUCCESS] Service started  
  
[SUCCESS] Service stopped  
  
[INFO  ] === exited ===
```


main.c — Controller & Logger

Fj: main.c — Controller and Logger Subsystem Architecture

CS-411 Project Report | Team Ahmad | April 2026



POSIX System Calls Used

System Call	File	Purpose
<code>getuid()</code>	<code>main.c</code>	Verify process runs as root (UID == 0)
<code>fork()</code>	<code>aleena.c</code>	Create child process to run system commands
<code>execvp()</code>	<code>aleena.c</code>	Replace child image with <code>cp / systemctl</code>
<code>waitpid()</code>	<code>aleena.c</code>	Parent waits for child to finish; gets exit code
<code>access()</code>	<code>marwan.c</code>	F_OK: file exists? X_OK: executable?
<code>fopen()</code>	<code>main.c / marwan.c</code>	Open <code>log.txt</code> (append) and service file (write)
<code>fprintf()</code>	<code>main.c / marwan.c</code>	Write unit file content and log entries
<code>time()</code> / <code>strftime()</code>	<code>main.c</code>	Generate [YYYY-MM-DD HH:MM:SS] timestamps
<code>strerror()</code>	all files	Human-readable error from <code>errno</code> for all failures

fork() + execvp() — How It Works

- 1 aleena.c calls `run_command(argv[])`
- 2 `fork()` — OS creates a child process
- 3 Child: `execvp()` replaces itself with `systemctl`
- 4 Child IS now `systemctl` — runs the command
- 5 Parent: `waitpid()` blocks until child finishes
- 6 `WEXITSTATUS()` extracts exit code (0 = success)

Why NOT `system()`?

- ✗ Spawns `/bin/sh` (extra overhead)
- ✗ Vulnerable to shell injection
- ✗ Limited error control
- ✗ Discouraged in POSIX programs
- ✓ `fork+exec`: no shell needed
- ✓ Args as array: injection-safe
- ✓ Full exit code via `waitpid()`
- ✓ Standard OS programming

What Happens in the Background

After './service_maker start myapp' — what does the OS actually do?

Step 1

systemd reads /etc/systemd/system/myapp.service into memory

Step 2

systemd forks a new child process (fork())

Step 3

Child calls exec() on ExecStart path — becomes your program

Step 4

systemd monitors the child — tracks PID, CPU, memory

Step 5

stdout/stderr captured by journal (viewable via journalctl)

Step 6

If process exits + Restart=always → systemd waits RestartSec then relaunches

Step 7

Symlink in multi-user.target.wants/ ensures auto-start on every boot

CLI Commands & Usage

CREATE

```
sudo ./service_maker create myapp /usr/local/bin/myservice.sh
```

→ [SUCCESS] Service 'myapp' enabled. ✓ Ready.

START

```
sudo ./service_maker start myapp
```

→ [SUCCESS] Service 'myapp' started.

STATUS

```
sudo ./service_maker status myapp
```

→ Active: active (running) • PID: 1234

STOP

```
sudo ./service_maker stop myapp
```

→ [SUCCESS] Service 'myapp' stopped.

No arguments? → Interactive menu appears: 1. Create 2. Start 3. Stop 4. Status 5. Exit

Live Demo — Create a Real Service

We create a shell script that writes a heartbeat log every 5 seconds, then register it as a service:

Step 1 — Create the script

```
sudo nano /usr/local/bin/myservice.sh  
#!/bin/bash  
while true; do  
    echo "Service is alive - $(date)" >> /var/log/myservice.log  
    sleep 5  
done
```

Step 2 — Make executable + Register

```
sudo chmod +x /usr/local/bin/myservice.sh  
sudo ./service_maker create myapp /usr/local/bin/myservice.sh
```

Step 3 — Start and Watch

```
sudo ./service_maker start myapp  
tail -f /var/log/myservice.log
```

Output every 5 seconds: Service is alive – Tue Apr 21 09:45:00 PDT 2026

Live Demo — Create a Real Service

```
ahmad@ubuntu: ~/cse302/files
ahmad@ubuntu:~/cse302/files$ sudo ./service_maker create myapp /usr/local/bin/myservice.sh

System Service Maker v1.0
Team Ahmad — CS-411
Aleena Khan
Marwan Rahim

— CREATE —
[INFO] Validating service name: myapp
[INFO] Validating executable path: /usr/local/bin/myservice.sh
[WARN] Local file 'myapp.service' already exists. Overwriting...
[SUCCESS] Service file created: myapp.service
[INFO] Installing service: myapp
[SUCCESS] Service file installed: /etc/systemd/system/myapp.service
[INFO] Reloading systemd daemon...
[SUCCESS] systemd daemon reloaded.
[INFO] Enabling service: myapp
[SUCCESS] Service 'myapp' enabled.

✓ Service 'myapp' is ready. Use 'start' to launch it.

ahmad@ubuntu:~/cse302/files$
```

Live Demo — Start a Real Service

```
ahmad@ubuntu: ~/cse302/files

ahmad@ubuntu:~/cse302/files$ sudo ./service_maker

System Service Maker v1.0
Team Ahmad - CS-411
Aleena Khan
Marwan Rahim

Usage:
sudo ./service_maker create <name> <exec_path>
sudo ./service_maker start <name>
sudo ./service_maker stop <name>
sudo ./service_maker status <name>

Examples:
sudo ./service_maker create myapp /usr/local/bin/myapp
sudo ./service_maker start myapp
sudo ./service_maker status myapp
sudo ./service_maker stop myapp

Select an operation:
1. Create service
2. Start service
3. Stop service
4. Status of service
5. Exit

Choice: 2
Service name: myapp

-- START --
[INFO] Starting service: myapp
[SUCCESS] Service 'myapp' started.
ahmad@ubuntu:~/cse302/files$
```

Live Demo —Check Status of a Real Service

```

ahmad@ubuntu: ~/cse302/files
sudo ./service_maker stop <name>
sudo ./service_maker status <name>

Examples:
sudo ./service_maker create myapp /usr/local/bin/myapp
sudo ./service_maker start myapp
sudo ./service_maker status myapp
sudo ./service_maker stop myapp

Select an operation:
1. Create service
2. Start service
3. Stop service
4. Status of service
5. Exit

Choice: 4
Service name: myapp

--- STATUS ---
[INFO] Checking status: myapp

● myapp.service - Auto Generated Service - myapp
   Loaded: loaded (/etc/systemd/system/myapp.service; enabled; vendor preset: enabled)
   Active: active (running) since Sat 2026-04-25 00:46:22 PDT; 42min ago
     Main PID: 962 (myservice.sh)
       Tasks: 2 (limit: 8723)
      Memory: 652.0K
         CPU: 4.959s
      CGroup: /system.slice/myapp.service
              └─ 962 /bin/bash /usr/local/bin/myservice.sh
                 13741 sleep 5

Apr 25 00:46:22 ubuntu systemd[1]: Started Auto Generated Service - myapp.

[SUCCESS] Service 'myapp' is ACTIVE.
ahmad@ubuntu: ~/cse302/files$

```

```

ahmad@ubuntu: ~/cse302/files$ tail -f /var/log/myservice.log
Service is alive - Sat Apr 25 01:27:47 AM PDT 2026
Service is alive - Sat Apr 25 01:27:52 AM PDT 2026
Service is alive - Sat Apr 25 01:27:57 AM PDT 2026
Service is alive - Sat Apr 25 01:28:02 AM PDT 2026
Service is alive - Sat Apr 25 01:28:07 AM PDT 2026
Service is alive - Sat Apr 25 01:28:12 AM PDT 2026
Service is alive - Sat Apr 25 01:28:17 AM PDT 2026
Service is alive - Sat Apr 25 01:28:22 AM PDT 2026
Service is alive - Sat Apr 25 01:28:27 AM PDT 2026
Service is alive - Sat Apr 25 01:28:32 AM PDT 2026
Service is alive - Sat Apr 25 01:28:37 AM PDT 2026
Service is alive - Sat Apr 25 01:28:42 AM PDT 2026
Service is alive - Sat Apr 25 01:28:47 AM PDT 2026
Service is alive - Sat Apr 25 01:28:52 AM PDT 2026
Service is alive - Sat Apr 25 01:28:57 AM PDT 2026
Service is alive - Sat Apr 25 01:29:02 AM PDT 2026
Service is alive - Sat Apr 25 01:29:07 AM PDT 2026
Service is alive - Sat Apr 25 01:29:12 AM PDT 2026
Service is alive - Sat Apr 25 01:29:17 AM PDT 2026
Service is alive - Sat Apr 25 01:29:22 AM PDT 2026
Service is alive - Sat Apr 25 01:29:27 AM PDT 2026
Service is alive - Sat Apr 25 01:29:32 AM PDT 2026
Service is alive - Sat Apr 25 01:29:37 AM PDT 2026

```

Error Handling & Validation

Invalid Path

```
create myapp /usr/bin/doesnotexist
```

```
[ERROR] Path does not exist: /usr/bin/doesnotexist
```

Space in Name

```
create "my app" /usr/bin/python3
```

```
[ERROR] Invalid character ' ' in service name.
```

Special Char

```
create my@app /usr/bin/python3
```

```
[ERROR] Invalid character '@' in service name.
```

Duplicate

```
create myapp /usr/bin/python3 (2nd time)
```

```
[WARN] Service 'myapp' already exists in  
/etc/systemd/system/
```

Not Found

```
start ghostservice
```

```
[ERROR] Failed to start 'ghostservice' (exit 5).
```

No Root

```
./service_maker (no sudo)
```

```
[ERROR] This tool requires root privileges.
```

Test Cases & Results

Test ID	Description	Expected Output	Result
TC-01	Valid service creation	File created + installed + enabled	✓ PASS
TC-02	Invalid executable path	[ERROR] Path does not exist	✓ PASS
TC-03	Space in service name	[ERROR] Invalid character ' '	✓ PASS
TC-04	Special char in name (@)	[ERROR] Invalid character '@'	✓ PASS
TC-05	Duplicate service creation	[WARN] Already exists in systemd	✓ PASS
TC-06	Start valid service	[SUCCESS] Service started	✓ PASS
TC-07	Status — active service	Active: active (running)	✓ PASS
TC-08	Stop running service	[SUCCESS] Service stopped	✓ PASS
TC-09	Status — after stop	Active: inactive (dead)	✓ PASS
TC-10	Run without sudo	[ERROR] Requires root privileges	✓ PASS
TC-11	log.txt creation + entries	Timestamped entries appended	✓ PASS

Why This Project is Better

Aspect	Manual	Our Tool
Steps needed	5 separate	1 command
Validation	None	Full POSIX
Duplicate check	✗ None	✓ Built-in
Shell injection risk	✗ system()	✓ fork+exec
Logging	✗ None	✓ Timestamped
Error messages	✗ Silent fail	✓ Descriptive
Beginner friendly	✗ No	✓ Yes



POSIX Compliant

fork()+execvp() — same technique real shells use



Modular Design

3 files, each with one clear responsibility



Safe by Default

Validates everything before touching the system



Audit Logging

Every operation timestamped in log.txt

Team Contributions

Muhammad Ahmad

Testing & Documentation

- main.c — CLI controller, pipeline orchestration
- write_log() — timestamped logging system
- check_root() — root privilege enforcement
- Interactive menu mode
- All test cases (TC-01 to TC-11)
- Project report + this presentation

Marwan Rahim

Core Logic

- marwan.c — complete module
- validate_service_name() with POSIX isalnum()
- validate_path() with access() F_OK + X_OK
- create_service_file() with duplicate check
- fopen() + fprintf() unit file generation

Aleena Khan

System Integration

- aleena.c — complete module
- run_command() fork()+execvp() helper
- install_service() via cp
- reload_daemon(), enable_service()
- start_service(), stop_service(), status_service()
- All WEXITSTATUS() error handling

Conclusion

"Our tool automates the entire systemd service lifecycle validation, file generation, installation, and control using POSIX system calls and modular C, replacing five manual steps with a single command."